

CS571 Programming Languages

Spring 2026

10. Logic Programming and Prolog

Programming Paradigms

- **Imperative**: programmer instructs the machine what to do
 - **Procedural**: group instructions into procedures
 - **OO**: encapsulation, subtyping and inheritance
- **Declarative**: programmer declares properties of the desired result, but not how to compute it
 - **Functional**: program is a function
 - Based on lambda calculus, execution is by β -reduction
 - **Prolog**: program is a collection of facts and rules
 - Based on first-order predicate logic, execution is by deduction
 - Formally, SLD resolution (Selective Linear Definite clause resolution)



Background

First-Order Predicate Logic

Axioms

Axioms (truths)

- 0 is a natural number
- For all natural numbers x , $x + 1$ is a natural number
- Read more about [Peano axioms](#) for natural #s

Deductions

Is 2 a natural number?

- Yes
- 0 is a natural #
- Since 0 is a natural #, $0 + 1 = 1$ is a natural #
- Since 1 is a natural #, $1 + 1 = 2$ is a natural #

First-Order Predicate Concepts

Non-logic symbols

- **Constants/atoms**: a name or number
- **Predicate**: A symbol that takes one or more arguments and return true or false
- **Function**: A symbol that takes one or more arguments and returns a value
- **Variables**: a symbol representing an unknown value

First-Order Predicate Concepts

Logic symbols

- Quantifiers

$\forall$
forall

$\exists$
exists

- Connectives

$P \wedge Q$
P and Q

$P \vee Q$
P or Q

$\neg P$
not P

$P \rightarrow Q$
P implies Q

$P \rightarrow Q$ is **logically equivalent** to
 $\neg P \vee Q$

$P \leftarrow Q$
P is implied by Q

$P \leftrightarrow Q$
P if and only if Q

Axioms in Predicate Logic

Axioms (truths)

- $\text{natural}(0)$
- $\forall x . \text{natural}(x) \rightarrow \text{natural}(x + 1)$

Deductions in Predicate Logic

Does $\text{natural}(2)$ hold?

- Yes

- $\text{natural}(0) \wedge [\forall x . \text{natural}(x) \rightarrow \text{natural}(x + 1)] \rightarrow \text{natural}(1)$

- $\text{natural}(1) \wedge [\forall x . \text{natural}(x) \rightarrow \text{natural}(x + 1)] \rightarrow \text{natural}(2)$

Deductions in Predicate Logic

Does **natural(-1)** hold?

- **Open world assumption**

- Maybe. The axioms don't imply it, but don't rule it out either

- **Closed world assumption**

- No, if it was true, it would be derivable from the axioms

Horn Clauses

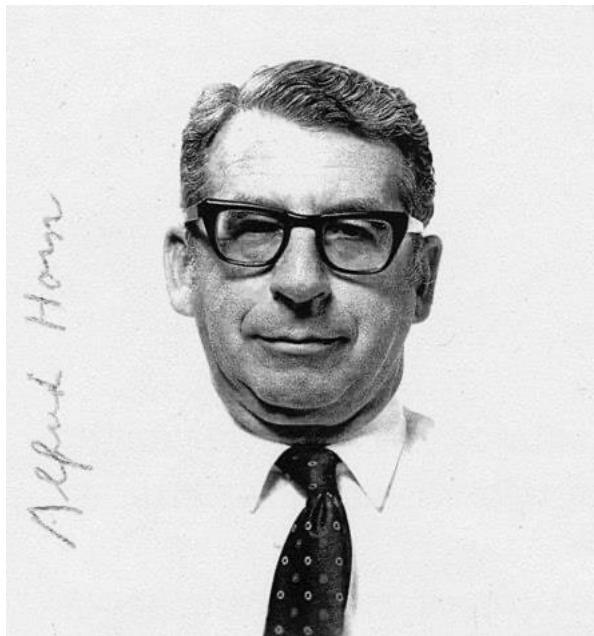


Image from Wikipedia

- Studied by Alfred Horn, published “On Sentences which are true of direct unions of algebras” in 1951
- Described what are now known as Horn Clauses:

$$\underbrace{a_1 \wedge a_2 \wedge \dots \wedge a_n}_{\text{conjunction}} \rightarrow \underbrace{b}_{\text{single conclusion}}$$

Quantification

- Variables that appear on **both sides** of the $\rightarrow$ are **universally quantified**

$$u > 0 \wedge v < 0 \rightarrow v < u$$

means $\forall u, v . u > 0 \wedge v < 0 \rightarrow v < u$

- Variables that appear **on only the left** of the $\rightarrow$ are **existentially quantified**

$$u > z \wedge v < z \rightarrow v < u$$

means $\forall u, v, \exists z . u > z \wedge v < z \rightarrow v < u$

Logical Programming Languages

A **logical programming language** is a **notational system** for writing logical statements, along with specified algorithms for implementing inference rules.

Prolog (Programming in logic)

- Originated from research in **Automated Theorem Proving**
- **Logic programming** first conceived of in Alain Colmerauer's research group in France
- First efficient implementation by David H. D. Warren in Scotland
- Used for theorem proving, automated planning, natural language processing, type systems...

Alain Colmerauer



Born	24 January 1941 Carcassonne , France
Died	12 May 2017 (aged 76) Marseille , France
Known for	Prolog

Image from Wikipedia

How to Run Prolog

- You can download and install Prolog locally:
 - <https://www.swi-prolog.org/>
- Or use the online version:
 - <https://swish.swi-prolog.org/>





Demo1

```
woman(mia) .
```

```
woman(jody) .
```

```
woman(yolanda) .
```

```
playsAirGuitar(jody) .
```

```
party.
```


Demo2

```
happy(yolanda) .
```

```
listens2Music(mia) .
```

- “:-” means $\leftarrow$ (implication)

Modus ponens (methods of affirming)

```
listens2Music(yolanda) :- happy(yolanda) .
```

```
playsAirGuitar(mia) :- listens2Music(mia) .
```

```
playsAirGuitar(yolanda) :- listens2Music(yolanda) .
```

Demo3

```
happy(vincent) .  
listens2Music(butch) .
```

```
playsAirGuitar(vincent) :-  
    listens2Music(vincent) ,  
    happy(vincent) .
```

```
playsAirGuitar(butch) :-  
    happy(butch) .
```

```
playsAirGuitar(butch) :-  
    listens2Music(butch) .
```

- “,” means **AND** ($\wedge$)
- “;” means **OR** ($\vee$)



Demo4

woman(mia) .

woman(jody) .

woman(yolanda) .

loves(vincent,mia) .

loves(marsellus,mia) .

loves(pumpkin,honey_bunny) .

loves(honey_bunny,pumpkin) .

Demo5

```
loves(vincent,mia) .
```

```
loves(marsellus,mia) .
```

```
loves(pumpkin,honey_bunny) .
```

```
loves(honey_bunny,pumpkin) .
```

```
jealous(X,Y) :- loves(X,Z) , loves(Y,Z) .
```

- Names that start with **capital letters** are **variables**

Elements in Prolog

Prolog data entities are referred to as **terms**. They consists of:

- Numbers
- Atoms
- Variables
- Structures (predicates)

Numbers

- Not really what Prolog is designed for, but Integers/Floats are supported
 - 10, 3.14, ...
 - Standard operators $+$, $-$, ...

Arithmetic

$$X < Y$$

$$X \leq Y$$

$$X = Y$$

$$X \neq Y$$

$$X \geq Y$$

$$X > Y$$

Prolog Syntax

$$X < Y$$

$$X = < Y$$

$$X =: = Y$$

$$X = \backslash = Y$$

$$X > = Y$$

$$X > Y$$

Atoms

- Identifiers beginning with a **lower-case** letter
 - `woman, mia`
- A sequence of special characters
 - `@ = ; : -`
- Any string within single quotes
 - `'mia loves music'`

Variables

Identifiers starting with **upper-case letters** or **underscores**

- E.g., X, Y, _9023, _
- Scope: all variables are local to the rule in which they occur. The same variables in two different rules are not the same
- Untyped: variables do not need to be declared. Prolog compilers are typically untyped
- Wildcard: _ can be used in place of a variable if you do not care its value. Unlike regular variables each occurrence of _ can match different terms

Structures (Predicates)

- **Predicates** are analogous to **procedures/functions** in imperative/functional programs
- Consist of a **functor** and a number of arguments

`woman(mia)` `married(mia, tim)`



- In a structure, arguments can be **constants**, **variables**, or **(nested) structures**
- **Arity**: number of arguments
 - `woman/1` has arity 1, `married/2` has arity 2

Functors

The functor is the outermost **atom** of a structure

- E.g., `student (ali, freshman, 194)`
- The functor of the above is `student`. It has arity 3.
 - Often written as `student/3`
- **Functors are NOT functions!**
 - But they can be used to define functions

Prolog Program

- A program in prolog is a collection of clauses which specify relationships between terms
 - Anything that ends with " . "
 - Consists of facts and rules
 - A program is also called a database or knowledge base

Facts

```
father(mike, susan) .  
father(mike, john) .  
mother(mary, susan) .  
mother(mary, john) .  
mother(jane, mary) .
```

Same as

```
father(mike, susan) :- true.  
...
```

Rules

`parent(X, Y) :- father(X, Y) .`

`parent(X, Y) :- mother(X, Y) .`

`grandparent(X, Y) :- parent(X, Z) , parent(Z, Y) .`

head

Implication symbol

condition

- Quantification
 - All variables in the **head** are **universally quantified**
 - All **others** are **existentially quantified**
- **AND/OR** semantics: `" , "` means **AND**, `" ; "` means **OR**
 - Different rules of the same predicate give the disjunctive meanings (**OR**)
 - `parent(X, Y) :- father(X, Y) ; mother(X, Y) .`
 - Conditions in the same rule separated by `,` are conjunctively formed (**AND**)

Queries

- A prolog program is run by posting queries to the database
 - Ask whether a relation holds between terms

```
father(mike, susan) .  
father(mike, john) .  
mother(mary, susan) .  
mother(mary, john) .  
mother(jane, mary) .
```

```
?- father(mike, susan) .  
true  
?- father(mike, tom) .  
false
```

Variables in Queries

- Any term can appear as a query, including a term with variables, which follow **existential** semantics
- Can be used to look up information

Does there exist an X such that `father(mike, X)` ?

```
?- father(mike, X) .  
X = susan %press ;  
X = john
```

- If more than one X matches, Prolog will return all answers
- Type “;” to ask for more answers
- If you don’t want more answers, simply press “**enter**”.

Conjunctive Queries

- Queries can be **ANDed** together too. The Prolog system tries to prove them all.

Who are **mike** and **mary's** children?

```
?- father(mike, X), mother(mary, X).  
X = susan %press ;  
X = john
```


Unification

- Almost all computation in Prolog is done using a general form of **pattern matching** called **unification**.
- Unification is an attempt to match two terms, by instantiating values in the terms.

$$\begin{array}{ccc} \text{rel}(X, b) & \longleftrightarrow & \text{rel}(a, Y) \\ & X = a & \\ & Y = b & \end{array}$$

$\text{rel}(a, b)$ is an **instance** of $\text{rel}(X, b)$ and $\text{rel}(a, Y)$

In general, two terms unify if they have a common instance

Rules for Unification

- A **constant** unifies only with itself
- Two structures unify only if they have the same **functor** and same **arity**, and the corresponding arguments unify **recursively**
- A variable unifies with anything
 - If the other thing has a value, the variable is **instantiated**
 - If the other thing is an uninstantiated variable, the two variables are **linked** such that if either is given a value later, the value is shared by both variables

When does Unification Happen?

- Explicitly with = operator:

```
rel(X, a) = rel(b, Y).
```

- Implicitly when applying rule:

```
identity(Z, Z). % rule
```

```
?- identity(rel(X, a), rel(b, Y)). %query
```

```
X = a
```

```
Y = b
```

" is " and " = "

- "=" means unification of structural forms
- "is" means **evaluating the right-hand side** of "is" as an arithmetic expression and **then performing unification**

?- 2*3 = 2*3.

?- 6 is 2*3.

?- 2*3 = 6.

?- 2*3 is 2*3.

?- 2*3 is 6.

" is " and " = "

- "=" means unification of structural forms
- "is" means **evaluating the right-hand side** of "is" as an arithmetic expression and **then performing unification**

```
?- 2*3 = 2*3.
```

```
true.
```

```
?- 2*3 = 6.
```

```
false.
```

```
?- 6 is 2*3.
```

```
true.
```

```
?- 2*3 is 2*3.
```

```
false.
```

```
?- 2*3 is 6.
```

```
false.
```

Review: Relations

- An **n-ary relation** R on sets $S_1, S_2, \dots, S_n$ is a set $R \subseteq (S_1 \times S_2 \times \dots \times S_n)$
 - $\text{People} = \{ \text{tom}, \text{jill}, \text{bill}, \text{mary}, \text{marge}, \dots \}$
 - $\text{Age} = \{ 0, 1, \dots, 120 \}$
 - $\{ (\text{tom}, 22), (\text{jill}, 45), (\text{bill}, 48) \}$ is a relation on $\text{People} \times \text{Age}$
 - $\{ (\text{tom}, \text{jill}), (\text{jill}, \text{mary}), (\text{bill}, \text{marge}) \}$ is a relation on $\text{People} \times \text{People}$
- The binary relation $(a, b) \in R$ is often written in **infix notation** as aRb .
 - E.g., $<$ is a relation on $\mathbb{R}$ ($< \subseteq \mathbb{R} \times \mathbb{R}$)
 - We write $2 < 3$ instead of $(2, 3) \in <$

Relations in Prolog

- Prolog facts are specifying relations
 - `{(mike, susan), (mike, john)}` is represented as a relation `father/2`

```
father(mike, susan) .  
father(mike, john) .
```

- Rules are also specifying relations, in terms of existing relations

Review: Functions

- A binary relation $f \subseteq (A \times B)$ is such that for every $a \in A$ there is at most one pair $(a, b) \in f$
 - Many-to-one or one-to-one, but not one-to-many
 - We usually write $f(a) = b$.
 - A is called the **domain** and B is called the **codomain**.
 - If the function f is defined for all $a \in A$, then it is said to be a **total function**. Otherwise, it is said to be a **partial function**.

Functions in Prolog

- Even though Prolog doesn't really have functions as a language construct, can define functions as relations
- In general, a function with n inputs and m outputs becomes a relation with $n + m$ arguments, where typically the inputs are given first, followed by the outputs

```
add_3_and_double(X,Y) :- Y is (X+3) * 2.
```

Example: Factorial

- If N is 0, then the result is 1
- Res is the factorial of N if
 - $N > 0$
 - $N1$ is value-equal to $N - 1$
 - Res1 is the factorial of $N1$
 - Res is value-equal to $N * \text{Res1}$

```
fact(0,1).  
fact(N, Res):-  
    N > 0,  
    N1 is N - 1,  
    fact(N1, Res1),  
    Res is N * Res1.
```

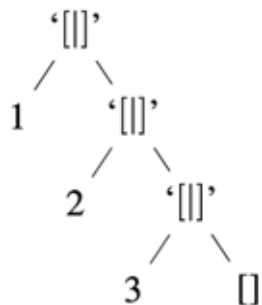
Example: Factorial

- `fact` is slow. Why?
- To make it more efficient, make it a **tail-recursive** function by using **accumulator**.

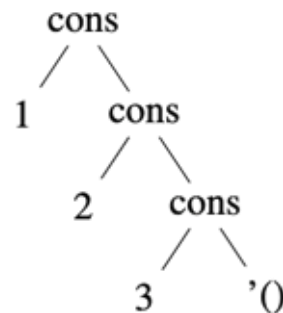
```
accFact(0,A,A) .  
accFact(N,A,Res) :-  
    N > 0,  
    N1 is N-1,  
    A1 is A*N,  
    accFact(N1,A1,Res) .  
Fact(N,Res) :  
    accFact(N,1,Res) .
```

List

- A **list** is
 - The atom `[]`, representing the empty list
 - A **compound term** with built-in functor `'[ | ]'`
 - Lists can be heterogeneous (Prolog is untyped anyways)



- Could be written using the standard syntax `'[]'(1, '[]'(2, '[]'(3, [])))`
- Or the equivalent form `[1 | [2 | [3 | []]]`
- Usually, written as `[1,2,3]`
- Same as how lists are defined in **Scheme**



List

- A non-empty list has a **head** and a **tail**
- A list with head **H** and tail **T** is denoted as **[H | T]**

?- [H | T] = [1, 2, 3] .

?- [X, Y, Z] = [1, 2, 3] .

?- [H | T] = [] .

?- [X, Y | Z] = [1, 2, 3] .

List

- A non-empty list has a **head** and a **tail**
- A list with head **H** and tail **T** is denoted as **[H | T]**

?- [H|T] = [1,2,3].

H = 1,

T = [2,3].

?- [X,Y,Z] = [1,2,3].

X = 1,

Y = 2,

Z = 3.

?- [H|T] = [].

false.

?- [X,Y|Z] = [1,2,3].

X = 1,

Y = 2,

Z = [3].

List

- A non-empty list has a **head** and a **tail**
- A list with head **H** and tail **T** is denoted as **[H | T]**

?- [X, [a, b], X|Z] = [a, [X|Z], a, b].

?- [X, [a, b], X|Z] = [a, [X, Z], a, b].

?- [X, [a, b], X|_] = [a, [X, _], a, b].

List

- A non-empty list has a **head** and a **tail**
- A list with head **H** and tail **T** is denoted as **[H | T]**

?- [X, [a, b], X|Z] = [a, [X|Z], a, b].

X = a,

Z = [b].

?- [X, [a, b], X|Z] = [a, [X, Z], a, b].

false.

?- [X, [a, b], X|_] = [a, [X, _], a, b].

X = a.

Example: List Functions

- Exercise: how to define **c(a/d)r** function (Scheme) in Prolog?

```
car ([H|_], H) .
```

```
cdr ([_|T], T) .
```

```
caar ([_, X|_], X) .
```

```
cadr ([[_|T]|_], T) .
```

Example: Length

- Finding the length of a list

- E.g., `length([1,2,3], X)` should return `X=3`.

Hint: what is the relationship between the length of `[1,2,3]` and the length of `[2,3]`?

Scheme

```
(define (len lst)
  (if (null? lst) 0
      (+ 1 (len (cdr lst)))))
```

Prolog

```
len([], 0).
len([_|T], N) :- len(T, N1), N is N1+1.
```

Example: Length

```
len([], 0) .
```

```
len([_ | T], N) :- len(T, N1), N is N1+1.
```

- Can you make length tail-recursive?

```
accLen([], A, A) .
```

```
accLen([_ | T], A, Len) :-
```

```
    A1 is A+1,
```

```
    accLen(T, A1, Len) .
```

```
len(Lst, Len) :- accLen(Lst, 0, Len) .
```

Example: Length

- Using the length function

```
?- len([1,2,3],3).
```

```
true.
```

```
?- len([1,2,3],X).
```

```
X = 3.
```

```
?- len(X,3).
```

```
?- len(X, 3) .  
X = [_ , _ , _] .
```

```
?- len(X,Y).
```

```
?- len(X, Y) .
```

```
X = [] ,
```

```
Y = 0 ;
```

```
X = [_] ,
```

```
Y = 1 ;
```

```
X = [_ , _] ,
```

```
Y = 2 ;
```

```
X = [_ , _ , _] ,
```

```
Y = 3 .
```

Example: Append

- Append two lists

```
?- append([1,2],[3,4],L) .
```

```
L = [1,2,3,4]
```

```
?- append(L,[3,4],[1,2,3,4]) .
```

```
L = [1,2]
```

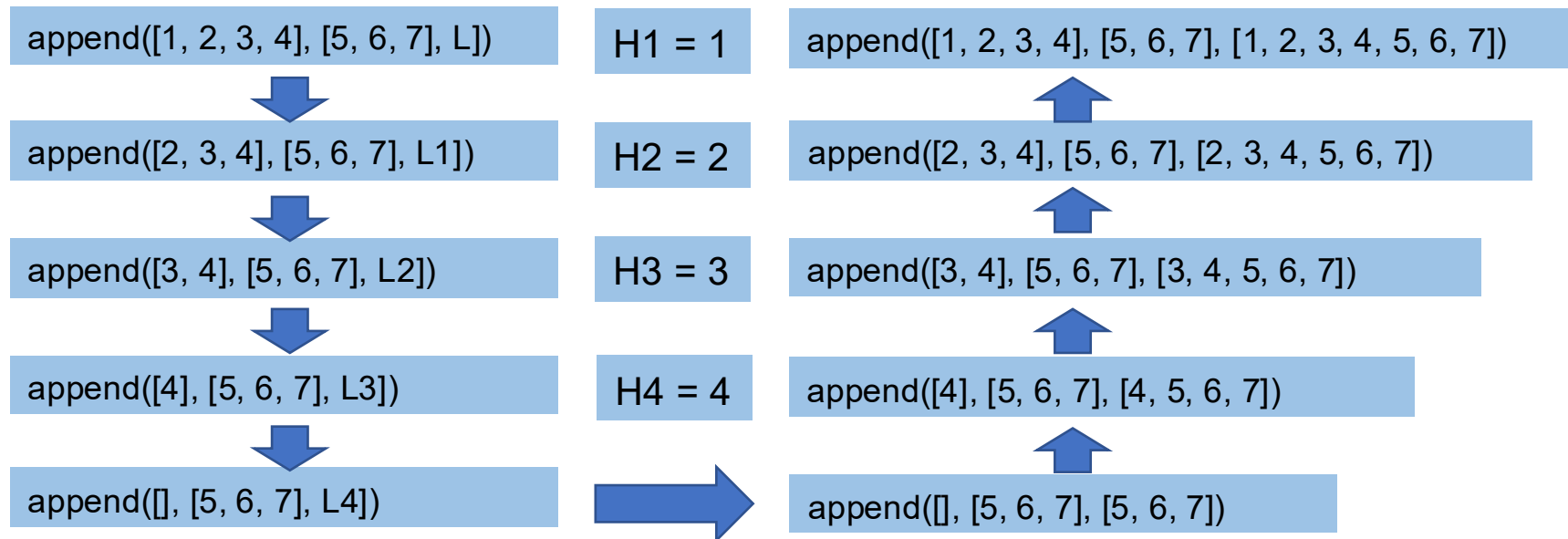
```
append([],L,L) .
```

```
append([H|T1],L,[H|T2]) :- append(T1,L,T2) .
```

Example: Append

`append([ ], L, L) .`

`append([H|T1], L, [H|T2]) :- append(T1, L, T2) .`



Example: Append

?- append(X, Y, [1, 2, 3]).

```
prefix(P, L) :- append(P, _, L).  
suffix(S, L) :- append(_, S, L).  
sublist(SubL, L) :-  
    suffix(S, L),  
    prefix(SubL, S).
```

```
X = [],  
Y = [1, 2, 3] ;  
X = [1],  
Y = [2, 3] ;  
X = [1, 2],  
Y = [3] ;  
X = [1, 2, 3],  
Y = [] ;  
false.
```

Example: Reverse

- Reversing a list

```
?- rev([1,2,3],X) .
```

```
X = [3,2,1] .
```

First attempt

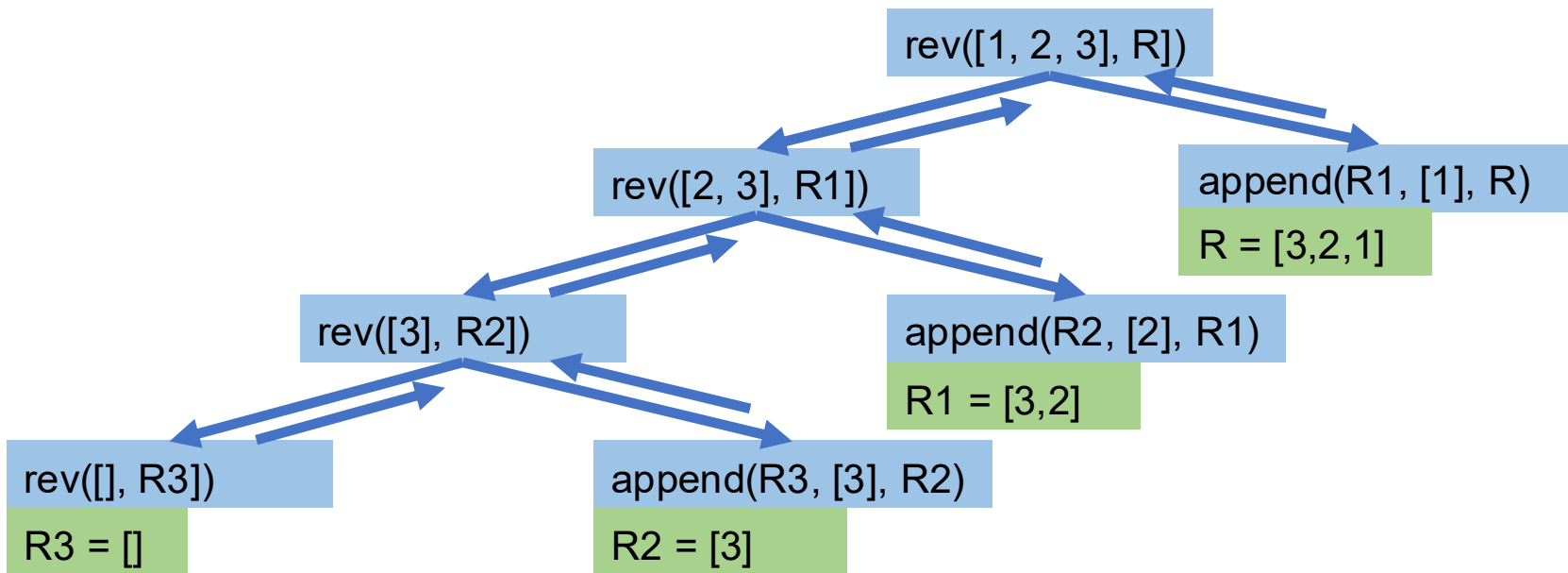
```
rev([], []).
```

```
rev([H|T],R) :- rev(T,R1), append(R1,[H],R) .
```


Example: Reverse

`rev([], []).`

`rev([H|T], R) :- rev(T, R1), append(R1, [H], R)`



Example: Reverse

```
rev([], []).
```

```
rev([H|T], R) :- rev(T, R1), append(R1, [H], R)
```

Second attempt (tail-recursive)

```
accRev([], A, A).
```

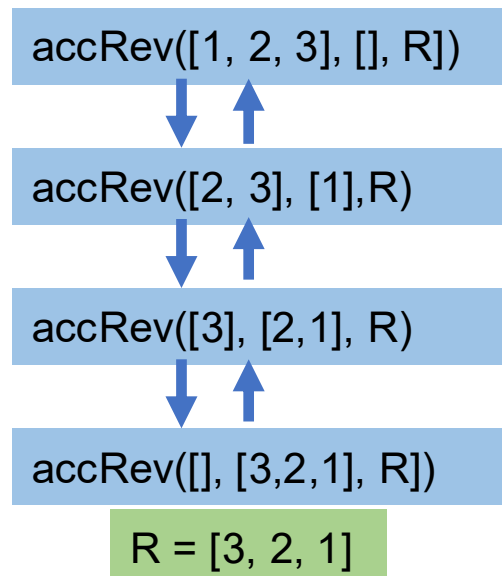
```
accRev([H|T], A, R) :- accRev(T, [H|A], R).
```

```
rev(L, R) :- accRev(L, [], R).
```

Example: Reverse

`accRev ([], A, A) .`

`accRev ([H|T], A, R) :- accRev (T, [H|A], R) .`



Prolog Semantics

Every Prolog program can be given two kinds of semantics:

- *Declarative* Semantics
 - The program is a collection of logical statements (facts and rules) about the world.
- *Operational* Semantics
 - A Prolog interpreter operates on the program to infer one or more goals from the program.

Declarative Semantics

The program is a collection of logical statements (facts and rules) about the world

- Each *fact* is unconditionally true
- Each *rule* is a logical implication: the conjunction of the *body* predicates implies the *head* predicate
- The *declarative meaning* of the program is the conjunction of the fact and rules in the program.
 - Ideally, each fact or rule is true about the domain, independent of any other rule
- The ordering of the facts/rules in a program **should not** affect the declarative meaning of the program
- The program implies further facts

Operational Semantics

A Prolog interpreter operates on the program to infer one or more goals from the program

- The operation of a Prolog System given a goal G is as follows:
 - The system finds the first rule or fact whose head matches G , and recursively attempts to solve all goals in the body with the substitution matching the head G applied to the body
 - If the attempt to solve the body succeeds, then the goal G succeeds
 - If solving the body fails, repeat with the next rule or fact matching G . If all such attempts fail, then the goal G fails
- This constitutes a depth-first search to satisfy the goal G (with backtracking)
- Not *complete* – meaning, not guaranteed to find a proof, even if a finite proof exists

Example

ancestor1.pl

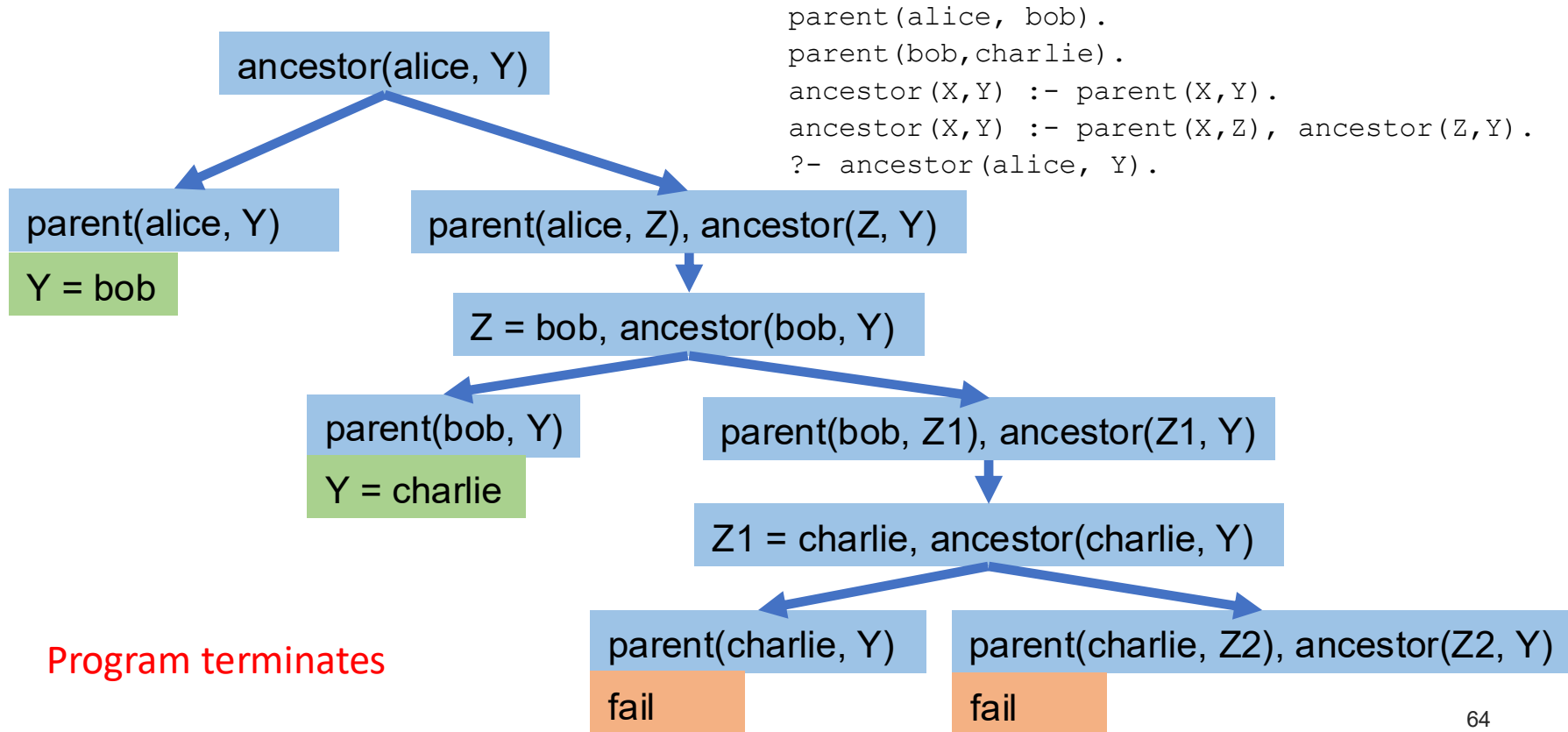
```
parent(alice, bob).  
parent(bob, charlie).  
ancestor(X,Y) :- parent(X,Y).  
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Are these two Prolog programs the same?

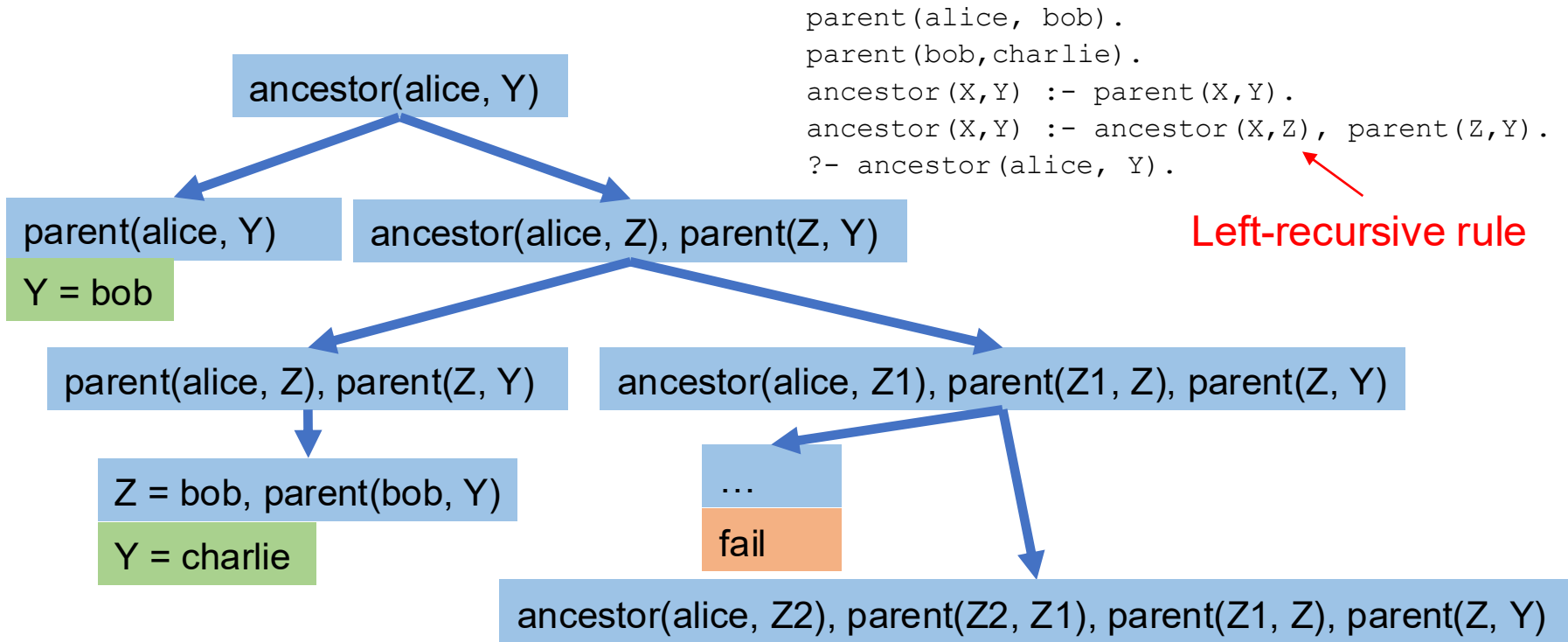
ancestor2.pl

```
parent(alice, bob).  
parent(bob, charlie).  
ancestor(X,Y) :- parent(X,Y).  
ancestor(X,Y) :- ancestor(X,Z), parent(Z,Y)
```

Example



Example



Program dose not terminate!

Programming Heuristics

Declaratively, order does not matter, but operationally it does matter.

- Write procedure with certain instantiation patterns in mind.
- Order facts/rules within a procedure to maximize the number of instantiation patterns for which program terminates.
 - Usually, facts/rules covering base cases before recursively-defined rules.
- Order the predicates within a body to ensure early failure.
 - Guards to ensure that a rule is applicable should be at the start of the body.
 - Avoid left-recursive rules.

Turing Completeness and Complexity



- Prolog is **Turing-complete** (as a subset of first-order predicate logic)
 - See below for a Turing machine simulation
https://www.swi-prolog.org/pack/file_details/turing/prolog/turing.pl?show=src
- Finding solutions for a Prolog program requires exponential time (**NP-hard**)

Not Covered

Some of the topics not covered include:

- The cut ! which allows control over backtracking
- Definite clause grammars: allows trivial construction of recursive-descent parsers
- Meta-predicates like assert, retract, call
- The relationship between Prolog and formal logic